Bisección y dividir y vencerás: soluciones

Programación: UVa 12032, UVa 10341 y UVa 10810

Carlos A Delgado S

Pontificia Universidad Javeriana Cali
Sesiones de refuerzo de algoritmia

Contenido

1 Bisección: encontrar la función

2 Bisección: cuando la función ya viene dada

3 Dividir y vencerás: plantear la recursión

Contenido

1 Bisección: encontrar la función
- Solución: UVa 12032

2 Bisección: cuando la función ya viene dada

3 Dividir y vencerás: plantear la recursión

La función y sus límites

La decisión

$\varphi(k)$: se simula la subida llevando la fuerza actual. Cada salto se compara contra ella, y el que la iguala la gasta en uno. Una pasada, $\Theta(n)$.

Por qué es monótona

Con $k+1$ la fuerza disponible es siempre mayor o igual que con k, así que ningún salto que cabía deja de caber.

Los límites

Por debajo, el salto más grande, que hay que dar. Por arriba, ese mismo más n: cada salto baja la fuerza a lo sumo en uno.

La decisión, en Python

```python
def alcanza(saltos, k):
    # Con fuerza inicial k, ¿se llega al último peldaño?
    fuerza = k
    puede = True
    for s in saltos:
        if s > fuerza:
            puede = False
        elif s == fuerza:
            fuerza = fuerza - 1
    return puede
```

El detalle

El ciclo no se corta cuando falla: deja la bandera en falso y sigue. Es lo mismo y respeta el único `return`.

La bisección, en Python

```
def minimaFuerza(saltos):
    # El menor k que cumple alcanza(saltos, k).
    lo = max(saltos)
    hi = lo + len(saltos)
    while lo < hi:
        medio = (lo + hi) // 2
        if alcanza(saltos, medio):
            hi = medio
        else:
            lo = medio + 1
    return lo
```

El intervalo

`lo` es el primer candidato que podría servir y `hi` el primero que seguro sirve. Cuando se juntan, ahí está el borde.

La misma decisión, en C++

```cpp
bool alcanza(const vector<int> &saltos, int k) {
  // Con fuerza inicial k, se llega al ultimo peldano?
  int fuerza = k;
  bool puede = true;
  for (size_t i = 0; i < saltos.size(); i++) {
    if (saltos[i] > fuerza) {
      puede = false;
    } else if (saltos[i] == fuerza) {
      fuerza = fuerza - 1;
    }
  }
  return puede;
}
```

La misma bisección, en C++

```cpp
int minimaFuerza(const vector<int> &saltos) {
  // El menor k que cumple alcanza(saltos, k).
  int lo = 0;
  for (size_t i = 0; i < saltos.size(); i++) {
    if (saltos[i] > lo) {
      lo = saltos[i];
    }
  }
  int hi = lo + (int)saltos.size();
  while (lo < hi) {
    int medio = lo + (hi - lo) / 2;
    if (alcanza(saltos, medio)) {
      hi = medio;
    } else {
      lo = medio + 1;
    }
  }
  return lo;
}
```

Dos detalles de la versión en C++

El punto medio

Se escribe `lo + (hi - lo) / 2` y no `(lo + hi) / 2`: con valores grandes la suma se desborda y la resta no.

El paso por referencia

Los saltos van como `const vector<int> &`: sin el `&` se copiaría el arreglo entero en cada una de las vueltas de la bisección.

Contenido

1 Bisección: encontrar la función

2 Bisección: cuando la función ya viene dada
- Solución: UVa 10341

3 Dividir y vencerás: plantear la recursión

Las dos comprobaciones

La monotonía

$f'(x) = -pe^{-x} + q\cos x - r\sin x + s\sec^2 x + 2tx$. En $[0, 1]$ cada sumando es menor o igual que cero por los rangos del enunciado, así que f no crece y hay a lo sumo una raíz.

Los extremos

Hay raíz exactamente cuando $f(0) \geq 0 \geq f(1)$. Si no, la respuesta es `No solution`, y hay que decidirlo antes de entrar al ciclo.

La solución en Python

```
def resolver(p, q, r, s, t, u):
    # f es decreciente en [0, 1]: hay raíz solo si f(0) >= 0 >= f(1).
    if f(p, q, r, s, t, u, 0.0) < 0.0 or f(p, q, r, s, t, u, 1.0) > 0.0:
        respuesta = 'No solution'
    else:
        lo = 0.0
        hi = 1.0
        vuelta = 0
        while vuelta < 100:
            medio = (lo + hi) / 2.0
            if f(p, q, r, s, t, u, medio) > 0.0:
                lo = medio
            else:
                hi = medio
            vuelta = vuelta + 1
        respuesta = '%.4f' % ((lo + hi) / 2.0)
    return respuesta
```

La solución en C++

```cpp
int main() {
  int p, q, r, s, t, u;
  while (scanf("%d %d %d %d %d %d", &p, &q, &r, &s, &t, &u) == 6) {
    if (f(p, q, r, s, t, u, 0.0) < 0.0 || f(p, q, r, s, t, u, 1.0) > 0.0) {
      printf("No solution\n");
    } else {
      double lo = 0.0, hi = 1.0;
      for (int vuelta = 0; vuelta < 100; vuelta++) {
        double medio = (lo + hi) / 2.0;
        if (f(p, q, r, s, t, u, medio) > 0.0) {
          lo = medio;
        } else {
          hi = medio;
        }
      }
      printf("%.4f\n", (lo + hi) / 2.0);
    }
  }
  return 0;
}
```

Las cien vueltas

Por qué un número fijo

Evita discutir con qué tolerancia parar: el intervalo queda en 2^{-100} del original, mucho más fino que los cuatro decimales que pide el enunciado.

La alternativa

Parar cuando `hi - lo` baje de 10^{-9} también sirve, pero obliga a elegir ese número y a confiar en que los redondeos no lo impidan.

Contenido

1 Bisección: encontrar la función

2 Bisección: cuando la función ya viene dada

3 Dividir y vencerás: plantear la recursión
- Solución: UVa 10810

El planteamiento

Las tres clases de parejas

Las de la mitad izquierda y las de la derecha son el mismo problema más chico. Las que cruzan se cuentan al mezclar: cuando sale un elemento de la derecha, todos los pendientes de la izquierda son mayores que él.

El costo

$T(n) = 2T(n/2) + \Theta(n)$, que da $\Theta(n \log n)$.

La combinación, en Python

```
def mezclar(a, aux, ini, medio, fin):
    # Mezcla a[ini:medio) con a[medio:fin) y cuenta las parejas que cruzan:
    # cada elemento que sale de la derecha aporta los que faltan a la
    ↪  izquierda.
    i = ini
    j = medio
    k = ini
    cruzadas = 0
    while k < fin:
        if i < medio and (j >= fin or a[i] <= a[j]):
            aux[k] = a[i]
            i = i + 1
        else:
            aux[k] = a[j]
            j = j + 1
            cruzadas = cruzadas + (medio - i)
        k = k + 1
    a[ini:fin] = aux[ini:fin]
    return cruzadas
```

La recursión, en Python

```
def contar(a, aux, ini, fin):
    # Ordena a[ini:fin) en el lugar y devuelve cuántas inversiones
    ↪  tenía.
    total = 0
    if fin - ini > 1:
        medio = (ini + fin) // 2
        total = contar(a, aux, ini, medio)
        total = total + contar(a, aux, medio, fin)
        total = total + mezclar(a, aux, ini, medio, fin)
    return total
```

Por índices

La función recibe `ini` y `fin` y trabaja sobre el mismo arreglo. El auxiliar se reserva una vez por caso.

La combinación, en C++

```cpp
long long mezclar(vector<int> &a, vector<int> &aux, int ini, int medio, int
↪  fin) {
  // Mezcla a[ini:medio) con a[medio:fin) y cuenta las parejas fuera de orden.
  int i = ini, j = medio, k = ini;
  long long cruzadas = 0;
  while (k < fin) {
    if (i < medio && (j >= fin || a[i] <= a[j])) {
      aux[k] = a[i];
      i = i + 1;
    } else {
      aux[k] = a[j];
      j = j + 1;
      cruzadas = cruzadas + (medio - i);
    }
    k = k + 1;
  }
  for (k = ini; k < fin; k++) {
    a[k] = aux[k];
  }
  return cruzadas;
}
```

La recursión, en C++

```cpp
long long contar(vector<int> &a, vector<int> &aux, int ini, int fin) {
  // Ordena a[ini:fin) en el lugar y devuelve cuantas inversiones
  ↪  tenia.
  long long total = 0;
  if (fin - ini > 1) {
    int medio = (ini + fin) / 2;
    total = contar(a, aux, ini, medio);
    total = total + contar(a, aux, medio, fin);
    total = total + mezclar(a, aux, ini, medio, fin);
  }
  return total;
}
```

El tipo del contador

`long long`: con medio millón de elementos la respuesta llega al orden de 10^{11} y no cabe en 32 bits.

Cuál enviar al juez

Medido con el tope del enunciado

Con $n = 499\,999$ la versión de C++ tarda 0,20 s y la de Python 5,1 s. Al juez va la de C++.

Lo que eso enseña

El orden de crecimiento es el mismo en los dos, $\Theta(n \log n)$. Lo que cambia es la constante, y cuando el límite de tiempo está ajustado la constante decide.